

The Open Source Audit

A Capstone Project — OSS NGMC Course

Software Under Audit:
The Linux Kernel

License: GNU General Public License v2.0

Student Name	Arpit Raj Singh
Registration No.	24BCE10118
Slot	A24
Date of Submission	31 March 2026

Table of Contents

1. Introduction
2. Part A — Origin and Philosophy
 - A1. The Problem Linux Was Created to Solve
 - A2. The License — What GPL v2 Actually Says
 - A3. The Ethics of Open Source
3. Part B — Linux Footprint on a Live System
4. Part C — The FOSS Ecosystem
5. Part D — Open Source vs Proprietary
6. Shell Script Tasks
7. Conclusion

1. Introduction

Most of the software powering the modern world wasn't built by a single company working behind closed doors. It was written in the open, shared freely, and shaped by thousands of contributors who never once met each other. The Linux Kernel stands as perhaps the most remarkable example of this phenomenon in the entire history of computing.

This report looks at the Linux Kernel through five lenses: where it came from and what problem it set out to solve, the specific freedoms its license gives and withholds, the ethical questions its very existence stirs up, its technical footprint on a live Linux system, and where it sits within the broader free and open-source software world. Alongside the written analysis, five shell scripts bring these ideas to life — demonstrating practical Linux skills rooted in the same spirit of transparency and automation that the kernel itself was built on.

But the Linux Kernel is more than a technical artifact. It's a philosophical statement — a declaration that foundational software infrastructure is too important to be locked away by any single company, and that the collective intelligence of a global community will consistently outrun any proprietary team. Understanding why Linus Torvalds made the choices he did back in 1991, and what those choices mean for every device running Linux today, is really what this audit is all about.

2. Part A — Origin and Philosophy

A1. The Problem the Linux Kernel Was Created to Solve

To understand why the Linux Kernel exists at all, you have to go back to the computing landscape of the late 1980s and early 1990s — a world where operating system kernels were either proprietary, expensive, or so educationally restricted that they were practically useless for anyone who actually wanted to learn by reading and modifying code.

The UNIX Dilemma

UNIX was the dominant professional operating system of that era — elegant, multi-user, multi-tasking, and the gold standard in universities and research labs. But UNIX was owned, first by AT&T; and later by SCO, and sold under a commercial license. Universities could get copies, but students had no real right to read the source code, modify it, or redistribute anything they built on top of it. The knowledge lived inside the system, and the system was locked.

Richard Stallman had spotted this problem earlier and responded with the GNU Project, launched in 1983. GNU aimed to build a completely free UNIX-compatible system — free not in the sense of price, but in the sense of freedom: the freedom to run, study, modify, and redistribute. By the early 1990s, GNU had produced an impressive set of tools — compilers, editors, shell utilities — but it was still missing one critical piece: a working kernel. The GNU Hurd kernel was in development but wouldn't be usable for years.

MINIX and the Educational Straitjacket

For university computer science courses, the go-to teaching operating system was MINIX, created by Professor Andrew Tanenbaum. MINIX was designed to be readable and understandable — a pedagogical UNIX clone. But Tanenbaum had licensed MINIX in a way that made it effectively read-only for practical purposes. Students could study it, but they couldn't freely modify and redistribute their changes. It was a teaching tool deliberately kept as a teaching tool, nothing more.

Linus Torvalds was a 21-year-old computer science student at the University of Helsinki in 1991. He had a personal computer — an Intel 386 — and he wanted to use it to explore UNIX properly. He couldn't afford a commercial UNIX license, and MINIX was the closest thing available to him. But it frustrated him. The terminal emulator he needed to connect to his university's network was broken in ways he simply couldn't fix, because MINIX's constraints prevented him from properly integrating his improvements into the system.

The Announcement That Changed Everything

On 25 August 1991, Torvalds posted a message to the comp.os.minix newsgroup that would become one of the most consequential paragraphs in computing history. He described a free operating system — a hobby project, he called it, explicitly saying he didn't expect anything big or professional like

GNU. He was just building something for 386 and 486 AT computers.

The choices Torvalds made in that early period deserve careful attention. He chose to write a monolithic kernel rather than the microkernel architecture Tanenbaum considered architecturally superior. He chose to target real, affordable consumer hardware rather than abstract ideal systems. And — most consequentially of all — he chose to release his kernel under the GNU General Public License in 1992, after initially using a more restrictive custom license. That single licensing decision opened the project to the full force of the GNU ecosystem and the growing community of developers who shared Stallman's philosophy.

Why This Mattered

What Torvalds solved wasn't just a personal inconvenience. He solved a structural problem in the computing world of 1991: the absence of a free, modifiable, redistributable kernel that could run on ordinary consumer hardware. The combination of the Linux Kernel with the GNU tools created what is now called GNU/Linux — the first complete free operating system stack that anyone in the world could run, study, modify, and redistribute without paying a license fee or signing a restrictive agreement.

This was not merely a convenience. It democratized access to foundational computing infrastructure for students in countries that couldn't afford commercial UNIX licenses, for researchers who needed to modify low-level behavior, for companies that needed a production-quality OS without vendor lock-in, and ultimately for the entire internet — which runs almost entirely on Linux servers today.

A2. The License — What GPL v2 Actually Says

The Linux Kernel is licensed under the GNU General Public License version 2, commonly known as GPL v2. This isn't a formality or a legal technicality. The choice of GPL v2 is a deliberate philosophical statement about how knowledge and infrastructure should be shared in society, encoded in legal language that has been tested in courts worldwide.

The Four Freedoms of Free Software

The Free Software Foundation defines free software through four essential freedoms. Here's how GPL v2 as applied to the Linux Kernel holds up against all four:

- **Freedom 0** — The freedom to run the program for any purpose. GPL v2 imposes no restriction on how or where you run Linux. A company can deploy it on a million servers, a student can install it on a laptop, a researcher can run it on custom hardware. No use-case requires additional permission.
- **Freedom 1** — The freedom to study how the program works and to change it to suit your needs. This requires access to source code, and the GPL v2 mandates that source code be made available to anyone who receives the binary. You can read every line of the Linux Kernel source, trace exactly how a system call is handled, understand the memory management subsystem in full detail, and modify any of it.
- **Freedom 2** — The freedom to redistribute copies. You may take the Linux Kernel, burn it to a disc, put it on a server, or ship it inside a product and give it away or sell it. GPL v2 permits all of

this.

- **Freedom 3** — The freedom to distribute copies of your modified versions. If you improve the kernel, you may share your improvements. Critically, GPL v2 requires that you do — this is the copyleft provision.

All four freedoms are granted. The GPL v2 is one of the strongest implementations of all four in existence.

The Copyleft Requirement — Can Companies Modify and Sell Without Sharing?

This is the question that most clearly separates the GPL from permissive licenses like MIT or Apache. The answer is: no, they cannot — and this is intentional.

The GPL v2 contains a copyleft provision. If you distribute a binary that incorporates GPL v2-licensed code, you must make the corresponding source code available to the recipients of that binary, under the same GPL v2 terms. A company shipping a product using a modified Linux Kernel — an Android phone manufacturer, a network router vendor, a smart TV producer — is legally required by GPL v2 to make their kernel source code available to customers.

This is why major Android device manufacturers publish kernel source code on their developer websites. It isn't generosity — it's a legal requirement imposed by the GPL v2. Companies that have violated this requirement have faced enforcement actions by the Software Freedom Conservancy and the FSF.

There is, however, an important nuance around user-space applications. If a program runs on top of Linux without linking into the kernel, the GPL does not automatically propagate to that program. This is why proprietary applications can run on Linux without their source code being required to be published.

GPL vs MIT — A Comparison

The MIT License is a permissive license. It lets anyone take the code, modify it, incorporate it into proprietary software, and distribute the result without sharing the source or applying any particular license to derivative work. The only requirement is attribution — keeping the original copyright notice intact.

The GPL is a copyleft license. It uses copyright law to ensure that the freedoms it grants cannot be stripped away by downstream recipients. Any derivative work distributed to others must carry the same GPL terms.

Dimension	GPL v2	MIT License
Can use in proprietary software?	No — copyleft propagates	Yes — no restrictions
Must share source code?	Yes — if distributing binaries	No
Can re-license derivatives?	No — must remain GPL	Yes — any license
Attribution required?	Yes	Yes

Dimension	GPL v2	MIT License
Best for infrastructure?	Yes — ensures freedom persists	Situational

If I were building something intended to be foundational infrastructure — a tool that others would rely on and build upon — I'd choose the GPL. The reason is philosophical: if the work is important enough to form the foundation of other systems, it's important enough to ensure that the freedom to study, modify, and improve it cannot be revoked by any downstream actor. Permissive licenses allow a company to take open work, add proprietary modifications, and lock the result away. The GPL prevents this.

If I were building a library intended for maximum adoption — including in commercial contexts — I might go with MIT or LGPL, accepting that some uses would be proprietary in exchange for wider reach.

Free as in Beer vs Free as in Freedom

This distinction, often attributed to Richard Stallman, is foundational to understanding open source philosophy. 'Free beer' means no monetary cost — you can have it without paying. 'Free as in freedom' (libre) means you have rights over the software: you can read it, change it, and share it.

Linux is both, but the second meaning is what actually matters. You can run Linux at zero cost, but that's almost incidental. What's significant is that you have the right to see exactly how every function in the kernel works, to modify the memory scheduler, to rewrite the file system layer, and to publish your version for others to use. A proprietary operating system may also be free of charge — Android without a carrier subsidy costs nothing to download — but you have no right to its source code, no ability to verify what it does, and no freedom to modify it. That's free beer, not free freedom.

A3. The Ethics of Open Source

Open source is often discussed in purely practical terms — reliability, security, cost. But at its core, it's an ethical position about how knowledge and tools should relate to power. The following reflections engage with the genuinely difficult questions that open source raises.

Should All Software Be Open Source?

The argument for universal open source is strong: software is knowledge, and the restriction of knowledge has historically been a tool of exclusion and control. When the source code of a medical device is proprietary, patients and doctors can't verify that it does what it claims. When a social media platform's algorithm is secret, citizens can't understand how their attention and opinions are being shaped. When educational software is closed, students learn to use tools without ever understanding them — building dependency rather than capability.

The counterargument is equally genuine: software development requires sustained investment. Companies that can't protect their innovations through proprietary licensing have reduced incentive to invest in expensive research and development. Some categories of software — security systems,

certain medical applications, national defence tools — may be dangerous if their internals are public knowledge. And the argument that 'openness guarantees security' isn't universally true; the Heartbleed vulnerability sat in open-source OpenSSL for years before anyone caught it.

My own view is that the strongest argument for open source applies most forcefully to infrastructure software — the layers that others build on top of and that affect everyone who uses technology. The kernel, the compiler, the network stack, the cryptographic libraries — these should be open, not because openness is always better, but because the risk of abuse from opacity is simply too high when the software is foundational. Application-layer software — a commercial word processor, a game, a proprietary business tool — presents a far less clear-cut case.

The Ethics of Corporate Exploitation of Open Source Labor

This is perhaps the most uncomfortable question in open source ethics, and it's one that has grown more pressing as major technology corporations have grown wealthy on open source foundations.

Red Hat was acquired by IBM for approximately 34 billion dollars in 2019. Its product is almost entirely built on Linux, which is developed by thousands of unpaid volunteers alongside a smaller number of paid contributors. Google's Android, running on over two billion devices, is built on the Linux Kernel. Meta operates data centers of enormous scale on Linux. Amazon Web Services, the world's largest cloud provider, runs predominantly on Linux. These companies generate revenues measured in hundreds of billions of dollars annually — on infrastructure they did not primarily create and do not primarily maintain.

Is this ethical? One view is that it's not merely ethical but is the intended outcome: the GPL permits this use precisely because widespread deployment multiplies the software's social utility. Restricting commercial use would reduce Linux's reach and impact. The GPL's requirement to publish modifications means companies must give back their kernel-level improvements, which benefits everyone.

A more critical view notes that there's an asymmetry: the individuals who contribute the most to the kernel are often engineers employed by these same large corporations, while the volunteers and smaller contributors who maintain less prestigious but essential components are often compensated inadequately — or not at all. The 'open source is a gift to humanity' narrative can be used to extract value from people who deserve compensation. The collapse of community-maintained projects through maintainer burnout — a genuine and well-documented phenomenon — is in part a consequence of the gap between the value extracted from open source and the support given to its human maintainers.

Standing on the shoulders of giants is uniquely possible in open source in a way that closed source prevents: you can see the giants, understand them, and build higher. But it also carries a responsibility — both legal, under the GPL, and moral — to contribute back to the ecosystem that made your vantage point possible.

3. Part B — Linux Footprint on a Live System

This section documents the Linux Kernel's presence on a live system running WSL2 (Windows Subsystem for Linux) with kernel version 6.6.87.2-microsoft-standard-WSL2 on an x86_64 architecture. WSL2 runs a genuine Linux kernel as a lightweight VM inside Windows, and all observations below were made on this running installation.

Installation Method

The Linux Kernel is available through the package manager of virtually every major Linux distribution. On Debian/Ubuntu-based systems, the kernel is installed via the apt package manager:

```
sudo apt install linux-image-$(uname -r)
```

On RPM-based systems (Fedora, RHEL, CentOS), the equivalent is:

```
sudo dnf install kernel
```

For custom kernel builds — which kernel developers and advanced users perform — the process involves downloading the source from kernel.org, configuring it with 'make menuconfig', compiling with 'make -j\$(nproc)', and installing with 'make modules_install && make install'. Depending on the hardware, this process can take anywhere from 30 minutes to several hours.

Key Directories

Directory / File	Purpose
/boot/vmlinuz-*	The compressed kernel binary loaded by the bootloader (GRUB) at startup
/boot/initrd.img-*	Initial RAM disk — a temporary root filesystem used during early boot
/lib/modules/\$(uname -r)/	Loadable kernel modules — device drivers, filesystems, network protocols
/proc/	Virtual filesystem exposing real-time kernel state (processes, memory, CPU)
/sys/	Sysfs — exposes device and driver information from the kernel to user space
/dev/	Device files — every device on the system is represented here by the kernel
/etc/modprobe.d/	Configuration for kernel module loading — blacklists, options, aliases
/var/log/kern.log	Kernel messages log — hardware events, driver errors, boot messages

User and Group — Security Model

The Linux Kernel itself runs with the highest privilege level on the system — kernel space, or ring 0 in x86 terminology. It doesn't run as a Linux user in the traditional sense; it runs before and outside the user model, because the user model is itself implemented by the kernel.

This matters enormously for security. The kernel/user space boundary is the most important security boundary in a Linux system. User-space processes — even those running as root — can't directly access kernel memory or execute privileged instructions without passing through the kernel's system call interface. A vulnerability that allows a process to execute code in kernel space is the most severe class of Linux vulnerability, because it bypasses all user-level security controls.

Service Management

The Linux Kernel isn't managed as a service the way user-space applications are. It's loaded at boot by the bootloader and runs until shutdown. The kernel version in use can be verified with:

```
uname -r # kernel version  uname -a # full system information
```

Kernel modules — the loadable components that add driver and feature support — can be managed at runtime using `lsmod` (lists currently loaded modules), `modprobe` (loads a module and its dependencies), `rmmod` (removes a loaded module), and `modinfo` (displays information about a module including its license).

Update Model

The Linux Kernel is updated through the distribution's package manager. On Ubuntu, 'sudo apt upgrade' installs new kernel versions when they're released by the distribution. The old kernel is preserved — multiple kernel versions coexist in `/boot`, and the user selects which to boot from via GRUB.

Security patches for critical vulnerabilities are applied by the kernel security team and backported to stable kernel branches. The `kernel.org` website maintains the mainline kernel, multiple stable branches, and long-term support (LTS) branches that receive patches for years. Enterprise distributions like RHEL backport security fixes to even older kernel versions for longer support windows.

4. Part C — The FOSS Ecosystem

Dependencies — What Linux Relies On

Despite being a foundational layer, the Linux Kernel itself has dependencies — tools required to build and maintain it:

- **GCC / Clang** — the GNU Compiler Collection and LLVM Clang are the two compilers used to build the kernel. The kernel has been primarily built with GCC for decades; Clang support was added more recently and is now used by Android's kernel builds.
- **GNU Binutils** — the assembler, linker, and related tools that process compiled kernel object files into the final binary.
- **GNU Make** — the build system used to orchestrate kernel compilation across millions of lines of C code.
- **OpenSSL / libcrypto** — used for kernel module signing, a security mechanism that verifies module integrity.
- **Perl** — several kernel build scripts are written in Perl.
- **Python** — used by kernel configuration tools (menuconfig in later versions uses Python-based interfaces).

What Linux Has Enabled

The question of what Linux has enabled is almost easier to answer by listing what it hasn't. The following is a partial account of its direct progeny and dependents:

- **Android** — Google's Android operating system uses the Linux Kernel as its foundation. Every Android phone, tablet, and smart device in the world runs a Linux kernel. That's over three billion active devices.
- **ChromeOS** — Google's desktop operating system also uses the Linux Kernel, and since 2020 has supported running standard Linux applications alongside its own environment.
- **Docker and Kubernetes** — container technology, which has transformed how software is deployed and scaled, relies fundamentally on Linux kernel features: namespaces, cgroups, and overlay filesystems. Containers simply aren't possible without these kernel primitives.
- **Embedded systems** — the vast majority of embedded devices (routers, smart TVs, network switches, in-flight entertainment systems, automotive control units) run Linux thanks to its hardware support, stability, and zero licensing cost.
- **Supercomputers** — as of 2024, all 500 of the world's fastest supercomputers run Linux. This isn't coincidental; it's because Linux's source availability lets researchers optimize it for specific hardware architectures in ways that proprietary OS licenses prohibit.
- **The internet** — virtually every internet-facing server runs Linux. The web, email, DNS, and the routing infrastructure of the internet all run on Linux.

The Linux Ecosystem — Distributions and Forks

The Linux Kernel forms the foundation of hundreds of Linux distributions, each combining the kernel with different sets of user-space tools, package managers, and default configurations. Key examples include Ubuntu (desktop and server), Debian (the parent of Ubuntu, used extensively in servers and embedded systems), Fedora and RHEL (Red Hat's community and enterprise distributions), Arch Linux (a rolling-release distribution favored by advanced users), and Android (Google's mobile-focused kernel variant).

Most distributions track the upstream kernel and apply patches rather than forking it outright. True long-term forks include Android's kernel, which diverged significantly, and specialized kernels like the real-time PREEMPT-RT patch series, used in industrial and audio processing applications.

The LAMP Stack Connection

Linux is the 'L' in LAMP — the foundational web server stack consisting of Linux, Apache HTTP Server, MySQL, and PHP/Python/Perl. The LAMP stack has powered a significant fraction of the world's web applications for over two decades. Linux provides the operating system, process management, networking, and filesystem for the entire stack. Without a free, redistributable kernel, the LAMP stack as a freely deployable web platform simply wouldn't have been economically viable for small businesses and independent developers.

Community and Governance

The Linux Kernel is governed through a maintainer hierarchy headed by Linus Torvalds, who holds the role of Benevolent Dictator for Life (BDFL) and makes final decisions on what code enters the mainline kernel. Below Torvalds are dozens of subsystem maintainers, each responsible for a specific area of the kernel — networking, memory management, filesystems, specific hardware architectures.

The primary development mailing list is the Linux Kernel Mailing List (LKML), one of the highest-volume technical mailing lists in the world, carrying thousands of patches, reviews, and discussions per day. The Linux Foundation, a nonprofit organization, supports Linux development by employing kernel developers and providing infrastructure. Major corporate sponsors of kernel development include Intel, AMD, Samsung, Google, Red Hat, and IBM — companies whose financial interests align with the kernel's continued excellence.

Governance of the kernel is unusual in that it's simultaneously meritocratic (the best technical argument usually wins) and hierarchical (subsystem maintainers have final authority within their domain). Linus Torvalds has been notably willing to reject patches from large companies when they don't meet technical standards.

5. Part D — Open Source vs Proprietary

The most appropriate proprietary alternative to the Linux Kernel is the Windows NT kernel, which powers Microsoft Windows. Here's a comparison across key dimensions:

Dimension	Linux Kernel (GPL v2)	Windows NT Kernel (Proprietary)
Cost	Zero — no license fee for any use, including commercial deployment	Per-seat or per-server licensing; significant cost at scale (RHEL/Ubuntu support has its own cost, but the kernel itself is free)
Source Availability	Fully open — every line of code is readable at kernel.org	Closed — source shared only under restrictive NDAs with selected partners
Security Auditability	Anyone can audit the code; vulnerabilities are found by a global researcher community	Security research relies on binary analysis and Microsoft's internal processes; independent audit is impossible
Support and Reliability	Community support is extensive; commercial support available from Red Hat, Canonical, and others. Server uptime records routinely exceed years	Microsoft provides official support with defined SLAs; well-tested for desktop and enterprise Windows workloads
Freedom to Modify	Complete — you can modify the scheduler, file system, network stack, and distribute your modifications (with source)	None without an NDA; no ability to legally modify and redistribute
Community vs Corporate Control	Kernel decisions made by technical maintainers; no single company controls the kernel	Entirely controlled by Microsoft; strategic direction driven by commercial interests
Hardware Support	Extraordinary breadth — runs on x86, ARM, RISC-V, MIPS, PowerPC, mainframes, embedded systems	Primarily x86/x64; ARM support added for Surface devices and Windows on ARM
Ecosystem	Forms the foundation of the entire internet, cloud computing, Android, and supercomputing	Dominates desktop computing and enterprise environments using Microsoft's application stack

Overall Verdict

For server, cloud, embedded, and high-performance computing deployments, the Linux Kernel isn't merely a viable alternative to proprietary kernels — it's the superior choice on almost every dimension. Its zero cost, complete auditability, extraordinary hardware support, and the impossibility of vendor lock-in make it the rational choice for any organization that thinks carefully about its

technology infrastructure. The fact that it also powers the world's most critical computing infrastructure is the empirical validation of that rational choice.

For desktop computing, the comparison is more nuanced. Windows' advantage lies in application ecosystem compatibility and driver support for consumer peripherals — not in any architectural or freedom-related dimension. As the Linux desktop ecosystem matures — evidenced by the growth of gaming on Linux through Proton and the increasing quality of Linux desktop environments — even this advantage is narrowing.

Would I contribute to the Linux Kernel? Absolutely — though the contribution process is rigorous. Patches submitted to the kernel mailing list are reviewed to high technical standards. The kernel isn't a project for beginners, but it's one where contributions, once accepted, have the largest possible reach. Writing a patch that improves a kernel subsystem means improving something that runs on billions of devices simultaneously. That's a form of leverage no proprietary software project can offer a contributor.

Shell Script Tasks

All five scripts were written and tested on a Linux system. Each script is documented below with its code, an explanation of its purpose, and the shell scripting concepts it demonstrates.

Script 1 — System Identity Report

This script produces a formatted welcome screen that displays key information about the running Linux system, including kernel version, current user, system uptime, distribution name, and a message about the Linux Kernel's GPL v2 license.

Shell concepts demonstrated: variables, command substitution with `$()`, echo for formatted output, the source command to read `/etc/os-release`, and string concatenation.

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Arpit Raj Singh | Course: Open Source Software
# Purpose: Display a formatted system information screen for the Linux Kernel audit

# --- Student information (fill these in before submission) ---
STUDENT_NAME="Arpit Raj Singh"
SOFTWARE_CHOICE="Linux Kernel"

# --- Collect system information using command substitution ---
KERNEL=$(uname -r)           # Running kernel version
ARCH=$(uname -m)             # CPU architecture (e.g. x86_64)
USER_NAME=$(whoami)          # Current logged-in user
HOME_DIR=$HOME               # User's home directory
UPTIME=$(uptime -p)          # Human-readable uptime
CURRENT_DATE=$(date '+%A, %d %B %Y %H:%M') # Formatted date and time

# --- Get distro name from /etc/os-release if it exists ---
if [ -f /etc/os-release ]; then
    source /etc/os-release
    DISTRO="$NAME $VERSION_ID"
else
    DISTRO="Unknown Linux Distribution"
fi

# --- Display formatted output using echo ---
echo "=====
echo "      OPEN SOURCE AUDIT — SYSTEM IDENTITY REPORT"
echo "=====
echo "  Student      : $STUDENT_NAME"
echo "  Software      : $SOFTWARE_CHOICE"
```

```
echo "===== "  
echo "  Distribution: $DISTR0"  
echo "  Kernel Ver   : $KERNEL"  
echo "  Architecture: $ARCH"  
echo "  User          : $USER_NAME"  
echo "  Home Dir      : $HOME_DIR"  
echo "  Uptime        : $UPTIME"  
echo "  Date/Time     : $CURRENT_DATE"  
echo "===== "  
echo "  License       : GNU General Public License v2.0 (GPL-2.0) "  
echo "  Kernel Src    : https://kernel.org"  
echo "===== "
```

```
# Run Script 1  
bash script1_system_identity.sh  
=====   
                OPEN SOURCE AUDIT - SYSTEM IDENTITY REPORT  
=====   
Student       : Arpit Singh  
Software      : Linux Kernel  
=====   
Distribution:  Ubuntu 24.04  
Kernel Ver   :  6.6.87.2-microsoft-standard-WSL2  
Architecture:  x86_64  
User         :  arpit  
Home Dir     :  /home/arpit  
Uptime       :  up 12 minutes  
Date/Time    :  Tuesday, 31 March 2026 07:17  
=====   
License      :  GNU General Public License v2.0 (GPL-2.0)  
Kernel Src   :  https://kernel.org  
===== 
```

Script 2 — FOSS Package Inspector

This script checks whether a specified package is installed on the system, displays its version and license information, and uses a case statement to print a philosophical note about the package based on its name. It handles both RPM-based and Debian-based systems.

Shell concepts demonstrated: if-then-else, case statement, command-line package manager queries (dpkg-query and rpm), pipe with grep, and conditional command execution.

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Arpit Raj Singh | Course: Open Source Software
# Purpose: Inspect the Linux Kernel installation and print its details and a philosophy
note

echo "=== FOSS Package Inspector ==="
echo "Software: Linux Kernel"
echo ""

# --- Collect basic kernel info directly from the running system ---
KERNEL_VERSION=$(uname -r)
ARCH=$(uname -m)

# --- Detect environment: WSL2, native Linux (apt), or RPM-based ---
# if-then-else chain to handle all three cases

if uname -r | grep -qi "microsoft\|wsl"; then
    # WSL2: Microsoft's custom kernel — not a dpkg package, detected via uname
    echo "STATUS: Linux Kernel is ACTIVE on this system"
    echo ""
    echo "Environment   : WSL2 (Windows Subsystem for Linux)"
    echo "Kernel Type    : Microsoft WSL2 Linux Kernel (based on upstream Linux)"
    echo "Version        : $KERNEL_VERSION"
    echo "Architecture   : $ARCH"
    echo "License         : GNU General Public License v2.0 (GPL-2.0)"
    echo "Source Code     : https://github.com/microsoft/WSL2-Linux-Kernel"
    echo ""
    echo "Note: WSL2 uses a Microsoft-built kernel based on the upstream"
    echo "      Linux Kernel. It runs as a lightweight VM inside Windows."
    echo "      GPL v2 requires Microsoft to publish the full source code."

elif command -v dpkg-query &>/dev/null; then
    # Native Debian/Ubuntu — look up kernel package via dpkg
    EXACT="linux-image-$KERNEL_VERSION"
```

```
# Try exact package name first; fallback to any linux-image package
if dpkg-query -W "$EXACT" &>/dev/null 2>&1; then
    PACKAGE="$EXACT"
else
    PACKAGE=$(dpkg-query -W -f='${Package}\n' 'linux-image-*' 2>/dev/null | head -1)
fi

if [ -n "$PACKAGE" ]; then
    echo "STATUS: $PACKAGE is installed (Debian/Ubuntu)"
    echo ""
    dpkg-query -W -f='Package:    ${Package}\nVersion:    ${Version}\nStatus:
${Status}\n' "$PACKAGE"
else
    echo "STATUS: Kernel is running. Version: $KERNEL_VERSION"
fi

elif command -v rpm &>/dev/null; then
    # RPM-based system (Fedora, RHEL, CentOS)
    if rpm -q kernel &>/dev/null 2>&1; then
        echo "STATUS: kernel is installed (RPM-based system)"
        echo ""
        rpm -qi kernel | grep -E 'Version|License|Summary|Architecture'
    else
        echo "STATUS: Kernel running. Version: $KERNEL_VERSION"
    fi
fi

else
    echo "STATUS: Linux Kernel is running. Version: $KERNEL_VERSION"
fi

echo ""
echo "=== Open Source Philosophy Note ==="

# --- Case statement: philosophy note based on kernel version string ---
case "$KERNEL_VERSION" in
    *microsoft*|*Microsoft*|*WSL*|*wsl*)
        echo "Linux Kernel (WSL2): Even Microsoft ships the Linux Kernel inside Windows."
        echo "The GPL v2 required them to publish the full source – freedom enforced by
law."
        echo "Source: https://github.com/microsoft/WSL2-Linux-Kernel"
        ;;
    *generic*|*lowlatency*)
        echo "Linux Kernel: The foundation on which all modern open computing rests."
        echo "Built by thousands worldwide, licensed under GPL v2 – freedom in law."
        ;;
    *)
        echo "Linux Kernel: The foundation on which all modern open computing rests."
        echo "Built by thousands worldwide, licensed under GPL v2 – freedom in law."
        echo "Every line of source code is readable at: https://kernel.org"
        ;;
Esac
```

```

arpit@LAPTOP-KN0LGD3H:~$ # Run Script 3
bash script3_disk_auditor.sh

=====
                        DISK AND PERMISSION AUDIT REPORT
=====
DIRECTORY                                PERMS / OWNER / GROUP  SIZE
=====
/etc                                     drwxr-xr-x  root  root   4.3M
/var/log                               drwxrwxr-x  root  syslog 389M
/home                                  drwxr-xr-x  root  root   288M
/usr/bin                              drwxrwxr-x  root  root   94M
/tmp                                   drwxrwxrwt  root  root   24K
/proc                                  dr-xr-xr-x  root  root    0
/sys                                   dr-xr-xr-x  root  root    0
/dev                                   drwxr-xr-x  root  root    0

=== Linux Kernel-Specific Directory Check ===
FOUND      : /boot
              Permissions: drwxr-xr-x  root  root' | Size: 4.0K

FOUND      : /lib/modules/6.6.87.2-microsoft-standard-WSL2
              Permissions: drwxr-xr-x  root  root | Size: ~4- 186M

FOUND      : /proc/sys/kernel
              Permissions: dr-xr-xr-x  root  root | Size: 0

FOUND      : /etc/modprobe.d
              Permissions: drwxr-xr-x  root  root | Size: 28K

=== Currently Running Kernel ===
Version      : 6.6.87.2-microsoft-6t-standard-WSL2
Architecture : x86_64
Boot image   : /boot/vmlinuz-6.6.87.2-microsoft-standard-WSL2

```

Script 3 — Disk and Permission Auditor

This script loops through a list of important Linux system directories and reports the permissions, owner, group, and disk usage of each. It also checks for the Linux Kernel's key directories specifically and reports their status.

Shell concepts demonstrated: for loop over an array, if-then to check directory existence, ls -ld for permission strings, du for disk usage, awk to extract fields, and formatted output alignment.

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: Arpit Raj Singh | Course: Open Source Software
# Purpose: Audit key system directories for permissions, ownership, and disk usage

# --- Define the list of standard directories to audit ---
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/proc" "/sys" "/dev")

echo "=====
echo "          DISK AND PERMISSION AUDIT REPORT"
echo "=====
printf "%-22s %-22s %-8s\n" "DIRECTORY" "PERMS / OWNER / GROUP" "SIZE"
echo "-----"
```

```
# --- For loop: iterate over each directory in the array ---
for DIR in "${DIRS[@]}; do
    if [ -d "$DIR" ]; then
        # Use awk to extract permissions, owner, group from ls -ld output
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
        # Get human-readable size; suppress errors for virtual filesystems
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        printf "%-22s %-22s %-8s\n" "$DIR" "$PERMS" "${SIZE:-N/A}"
    else
        # Directory does not exist on this system
        printf "%-22s %s\n" "$DIR" "DOES NOT EXIST"
    fi
done

echo ""
echo "=== Linux Kernel-Specific Directory Check ==="

# --- Check directories specific to the Linux Kernel installation ---
KERNEL_DIRS=(
    "/boot"
    "/lib/modules/$(uname -r)"
    "/proc/sys/kernel"
    "/etc/modprobe.d"
)

for KDIR in "${KERNEL_DIRS[@]}; do
    if [ -d "$KDIR" ]; then
        PERMS=$(ls -ld "$KDIR" | awk '{print $1, $3, $4}')
        SIZE=$(du -sh "$KDIR" 2>/dev/null | cut -f1)
        echo "FOUND      : $KDIR"
        echo "          Permissions: $PERMS | Size: ${SIZE:-N/A}"
    else
        echo "NOT FOUND: $KDIR"
    fi
done
echo ""

# --- Display currently running kernel version as a summary ---
echo "=== Currently Running Kernel ==="
echo "Version      : $(uname -r)"
echo "Architecture : $(uname -m)"
echo "Boot image   : /boot/vmlinuz-$(uname -r)"
```

```
arpit@LAPTOP-KN0LGD3H:~$ # Run Script 3
bash script3_disk_auditor.sh

=====
                        DISK AND PERMISSION AUDIT REPORT
=====
DIRECTORY                                PERMS / OWNER / GROUP  SIZE
=====
/etc                                     drwxr-xr-x  root  root   4.3M
/var/log                               drwxrwxr-x  root  syslog 389M
/home                                  drwxr-xr-x  root  root   288M
/usr/bin                              drwxrwxr-x  root  root   94M
/tmp                                  drwxrwxrwt  root  root   24K
/proc                                  dr-xr-xr-x  root  root    0
/sys                                  dr-xr-xr-x  root  root    0
/dev                                  drwxr-xr-x  root  root    0

=== Linux Kernel-Specific Directory Check ===
FOUND      : /boot
              Permissions: drwxr-xr-x  root  root' | Size: 4.0K

FOUND      : /lib/modules/6.6.87.2-microsoft-standard-WSL2
              Permissions: drwxr-xr-x  root  root | Size: ~4- 186M

FOUND      : /proc/sys/kernel
              Permissions: dr-xr-xr-x  root  root | Size: 0

FOUND      : /etc/modprobe.d
              Permissions: drwxr-xr-x  root  root | Size: 28K

=== Currently Running Kernel ===
Version      : 6.6.87.2-microsoft-6t-standard-WSL2
Architecture : x86_64
Boot image   : /boot/vmlinuz-6.6.87.2-microsoft-standard-WSL2
```

Script 4 — Log File Analyzer

This script reads a log file line by line using a while-read loop, counts occurrences of a specified keyword, and prints a summary. It accepts the log file path and keyword as command-line arguments, with a default keyword of 'error'. It also prints the last five matching lines.

Shell concepts demonstrated: while read loop, if-then inside a loop, command-line arguments (\$1, \$2), counter variables, default argument values with \${VAR:-default}, file existence checking, and grep with tail for filtered output.

```
#!/bin/bash
# Script 4: Log File Analyzer
# Author: Arpit Raj Singh | Course: Open Source Software
# Usage: ./script4_log_analyzer.sh /path/to/logfile [keyword]
# Example: ./script4_log_analyzer.sh /var/log/kern.log error

# --- Accept command-line arguments ---
LOGFILE=$1                # First argument: path to the log file to analyze
KEYWORD=${2:-"error"}     # Second argument: keyword to search for (default: "error")
COUNT=0                  # Counter to track how many matching lines are found

# --- Validate that a log file argument was provided ---
```

```
if [ -z "$LOGFILE" ]; then
    echo "Usage: $0 <logfile> [keyword]"
    echo "Example: $0 /var/log/kern.log error"
    exit 1
fi

# --- Check that the log file exists and is a regular file ---
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File '$LOGFILE' not found or is not a regular file."
    exit 1
fi

# --- Check that the file is not empty before processing ---
if [ ! -s "$LOGFILE" ]; then
    echo "Warning: File '$LOGFILE' is empty. Nothing to analyze."
    exit 0
fi

echo "====="
echo "  LOG FILE ANALYZER"
echo "====="
echo "  Log File : $LOGFILE"
echo "  Keyword  : '$KEYWORD' (case-insensitive)"
echo "====="
echo ""

# --- Read the file line by line using a while-read loop ---
while IFS= read -r LINE; do
    # If the current line contains the keyword (case-insensitive), increment counter
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

# --- Count total lines in file using wc -l ---
TOTAL=$(wc -l < "$LOGFILE")

# --- Print the summary results ---
echo "  Total lines in file      : $TOTAL"
echo "  Lines matching '$KEYWORD' : $COUNT"

# --- Calculate match percentage if total is non-zero ---
if [ "$TOTAL" -gt 0 ]; then
    PERCENT=$(( COUNT * 100 / TOTAL ))
    echo "  Match percentage        : $PERCENT%"
fi

echo ""
echo "=== Last 5 lines matching '$KEYWORD' ==="

# --- Use grep to find matches, then tail to show the last 5 ---
MATCHES=$(grep -i "$KEYWORD" "$LOGFILE" | tail -5)
```

```
if [ -z "$MATCHES" ]; then
    echo "    (No matches found for '$KEYWORD')"
else
    echo "$MATCHES"
fi

echo ""
echo "Analysis complete."
```

```
arpit@LAPTOP-KNOLGD3H:~$ # Run Script 4 - needs a log file as input
bash script4_log_analyzer.sh /var/log/syslog error
=====
LOG FILE ANALYZER
=====
Log File : /var/log/syslog
Keyword  : 'error' (case-insensitive)
=====
Total lines in file : 69964
Lines matching 'error' : 127
Match percentage    : 0%

-- Last 5 lines mathing 'error' --
2026-03-31T18:04:35.740212+00:00 LAPTOP-KNOLGD3H kernel: WSL (240) ERROR: CheckConnection: getaddrinfo() failed: -5
2026-03-31T07:04:08.552877+00:00 LAPTOP-KNOLGD3H systemd[1]: apport-autoreport.path -
Process error reports when automatic reporting is enabled (file watch) was skipped because
of an unmet condition check (ConditionPathExists=/var/lib/apport/autoreport).
2026-03-31T07:04:08.555272+00:00 LAPTOP-KNOLGD3H systemd[1]: apport-autoreport.hemertimer -
Process error reports when automatic reporting is enabled (timer base) was skipped because
of an unmet condition check (ConditionPathExists=/var/lib/apport/autoreport).
2026-03-31T07:04:08.555272+00:00 LAPTOP-KNOLGD3H kernel: RAS: Correctable Errors initialized.
2026-03-31T07:04:07.305647+00:00 LAPTOP-KNOLGD3H kernel: WSL (248) ERROR: CheckConnection:
Getaddrinfo() failed: -5 chavnoed canese n repeaced check after it opens.
2026-03-31T07:04:06.585538+00:00 LAPTOP-KNOLGD3H kernel: WSL (248) ERROR: CheckConnect(on) failed: -5
2026-03-31T07:04:06.585538+00:00 LAPTOP-KNOLGD3H kernel: WSL (248) ERROR: CheckConnection: failed: -5
Analysis complete.
```

Script 5 — The Open Source Manifesto Generator

This script interactively asks the user three questions about their relationship with open source software, then composes a personalized philosophy statement using their answers and saves it to a text file. The script demonstrates the most creative use of shell features in this project.

Shell concepts demonstrated: read for interactive user input, string concatenation, writing to a file with `>` and `>>`, the date command, heredoc (`<<EOF`), command substitution, and the concept of aliases demonstrated through comments.

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Arpit Raj Singh | Course: Open Source Software
# Purpose: Generate a personalised open source philosophy statement
#          by asking the user three questions interactively and saving
```

```
#           the result to a text file.

# --- Alias concept demonstration (comment) ---
# In an interactive bash session, you could define:
# alias manifesto='bash script5_manifesto_generator.sh'
# Aliases are shorthand commands that expand to longer strings.
# We demonstrate the concept here in a comment as scripts don't persist aliases.

echo "=====
echo "      THE OPEN SOURCE MANIFESTO GENERATOR"
echo "=====
echo "  Answer the questions below to generate your manifesto."
echo "=====
echo ""

# --- Collect user input interactively using read -p ---
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you in software? " FREEDOM
read -p "3. Name one thing you would build and share freely with the world: " BUILD
read -p "4. Your name (for the manifesto signature): " NAME

# --- Use the date command to get the current date and time ---
DATE=$(date '+%d %B %Y')
TIME=$(date '+%H:%M')

# --- Build the output filename using string concatenation and whoami ---
OUTPUT="manifesto_$(whoami).txt"

echo ""
echo "Composing your manifesto..."
echo ""

# --- Write the file header using > (creates or overwrites the file) ---
echo "===== " > "$OUTPUT"
echo "      OPEN SOURCE MANIFESTO" >> "$OUTPUT"
echo "      Written by: $NAME" >> "$OUTPUT"
echo "      Date: $DATE at $TIME" >> "$OUTPUT"
echo "===== " >> "$OUTPUT"
echo "" >> "$OUTPUT"

# --- Compose the manifesto body using a heredoc (<<EOF) with variable substitution ---
# The heredoc writes multiple lines to the file in one block
cat >> "$OUTPUT" << EOF
I am a person who uses $TOOL every day – a tool I did not build alone,
but one that thousands of people around the world chose to share freely.
They did not have to. They chose to.

To me, freedom in software means $FREEDOM. Not as an abstraction,
but as a lived reality: the freedom to look inside the tools I depend on,
to understand them, to improve them, and to pass them on.

Linus Torvalds was a student who could not afford a Unix system. Richard
Stallman watched a printer fail and could not fix it because the code was
```

locked. The people who built the tools I use every day faced those same walls – and instead of accepting them, they built doors.

One day, I want to build \$BUILD and release it freely.
Not because the license requires it. Because I understand now what it means to stand on the shoulders of those who built in the open before me.

This is my commitment to open source.

– \$NAME, \$DATE
EOF

```
# --- Append a closing separator using >> ---
echo "" >> "$OUTPUT"
echo "===== " >> "$OUTPUT"
echo "  Generated by the OSS Capstone Manifesto Script" >> "$OUTPUT"
echo "  Linux Kernel Audit | Course: Open Source Software" >> "$OUTPUT"
echo "===== " >> "$OUTPUT"

# --- Display the saved manifesto to the terminal using cat ---
echo "===== "
echo "  Manifesto saved to: $OUTPUT"
echo "===== "
echo ""
cat "$OUTPUT"
```

```
arpit@LAPTOP-WHOLG034:~$ # Run Script 5 - interactive, it will ask you questions
bash script5_manifesto_generator.sh
=====
THE OPEN SOURCE MANIFESTO GENERATOR
=====
Answer the questions below to generate your your manifesto.
=====
1. Name one open-source tool you use every day: Linux
2. In your word, what does 'freedom' mean to you in In ssoftware? no tracking of my data
3. Name one thing you would build and share freely with the world: a free developer toolkit toolkit for Linux
4. Your name (for the manifesto signature): Arpit Raj Singh

Composing your manifesto... manifesto_arpit.txt
=====
OPEN SOURCE MANIFESTO
=====
Written by: Arpit Raj Singh
Date: 31 March 2026 at 07:41

I am a person who uses Linux every day – a tool I did not build alone,
but one that thousands of people around the world chose to share freely.
They did not have to. They chose to.

To me, freedom in software means no tracking of my data. Not as an abstraction,
but as a lived reality: the freedom to look inside the tools I depend on,
to understand them, to improve them, and to pass them on.

Linus Torvalds was a student who could not afford a Unix system. Richard
Stallman watched a printer fail and could not fix it because the code was
locked. The people who built the tools I use every day faced those same
walls – and instead of accepting them, they built doors.

One day, I want to build a free developer toolkit for Linux and release it freely.
Not because the license requires it. Because I understand now what it means
to stand on the shoulders of those who built in the open before me.

This is my commitment to open source.
– Arpit Raj Singh, 31 March 2026
=====
```

Conclusion

The Linux Kernel is not merely a piece of software. It is proof of a hypothesis: that a globally distributed, volunteer-led, openly licensed collaborative effort can produce software of higher quality, broader applicability, and greater durability than any proprietary alternative. The hypothesis was considered controversial in 1991. It is empirically settled today.

The origin story of the Linux Kernel is, at its core, a story about frustration with inaccessibility. Linus Torvalds wanted a tool he could not afford and could not modify if he could. His response was not to accept the limitation but to build the alternative and give it away. That act of giving — encoded in the GPL v2 and replicated by thousands of contributors since — is what transformed a student's hobby project into the operating system of the internet, the cloud, every Android device, and all of the world's fastest supercomputers.

Understanding the GPL v2 deeply changes how you think about software as property. Copyleft is not a restriction — it is a protection. It protects the freedom of every future user of the kernel from having that freedom revoked by any actor, however powerful. The distinction between 'free as in beer' and 'free as in freedom' is not a semantic game; it is the difference between access and right.

The ethical questions raised by open source — about corporate use of community labor, about whether all software should be open, about the responsibilities that come with standing on others' work — do not have clean answers. But asking them seriously is part of what it means to be a professional in a field where every tool you use, every language you write in, and every platform you deploy on was shaped by someone who chose to build in the open. That choice deserves to be understood, respected, and where possible, reciprocated.

The five shell scripts accompanying this report are themselves a small demonstration of the same principle: automation written transparently, with comments that explain not just what the code does but why. That commitment to clarity and explainability — the same commitment that makes open source auditable — is the most practically useful thing this course can instill.

References

Linux Kernel source code and documentation — kernel.org

GNU General Public License v2.0 full text — gnu.org/licenses/gpl-2.0.html

Free Software Foundation — The Free Software Definition — gnu.org/philosophy/free-sw.html

Open Source Initiative — The Open Source Definition — opensource.org/osd

Eric S. Raymond — The Cathedral and the Bazaar — catb.org/~esr/writings/cathedral-bazaar

Linus Torvalds — comp.os.minix newsgroup post, 25 August 1991

The Linux Command Line by William Shotts — linuxcommand.org

Linux Foundation annual report and contributor statistics — linuxfoundation.org

SPDX License List — spdx.org/licenses/GPL-2.0-only.html